

Practical Skills in Computational Biology

Fundamentals of Bioimage Processing - Unit 3: Image Segmentation

Anna Möller^{1, 2} Dr. Anne Hartebrodt¹ Prof. Dr. Stefan Uderhardt² Prof. Dr. David B. Blumenthal¹

¹Biomedical Network Science Lab, Department of Artificial Intelligence in Biomedical Engineering,
Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU),

²Department of Medicine 3 – Rheumatology and Immunology, FAU and Universitätsklinikum Erlangen, 91054, Germany

23.06.2026

1. Basic Concepts

2. Thresholding

3. Binary Operations

4. Connected Components and Labelling

5. Watershed

6. Evaluation of Segmentation Quality

7. Readings and References

- **Segmentation** partitions an image into meaningful regions.
- Goal: assign a label to every pixel $I[x, y]$ such that pixels sharing a label share a common property (intensity, texture, boundary, ...).
- Formal view – a segmentation is a function

$$S : \Omega \longrightarrow \mathcal{L}, \quad \Omega = \{(x, y) \mid x \in [0, W), y \in [0, H)\},$$

where $\mathcal{L}$ is the set of labels.

- **Two fundamental tasks**
 - *Semantic segmentation*: every pixel gets a class label (foreground / background, cell / nucleus / ...).
 - *Instance segmentation*: each individual object receives a unique label.

Semantic vs. Instance Segmentation



Nucleus image from [1].

- Downstream quantification requires object-level measurements:
 - Cell count, area, perimeter, elongation, fluorescence intensity per object.
- Serves as the bridge between raw image data and biological insight.
- Errors in segmentation propagate directly into any downstream analysis.
- Almost every bioimage analysis pipeline contains a segmentation step.

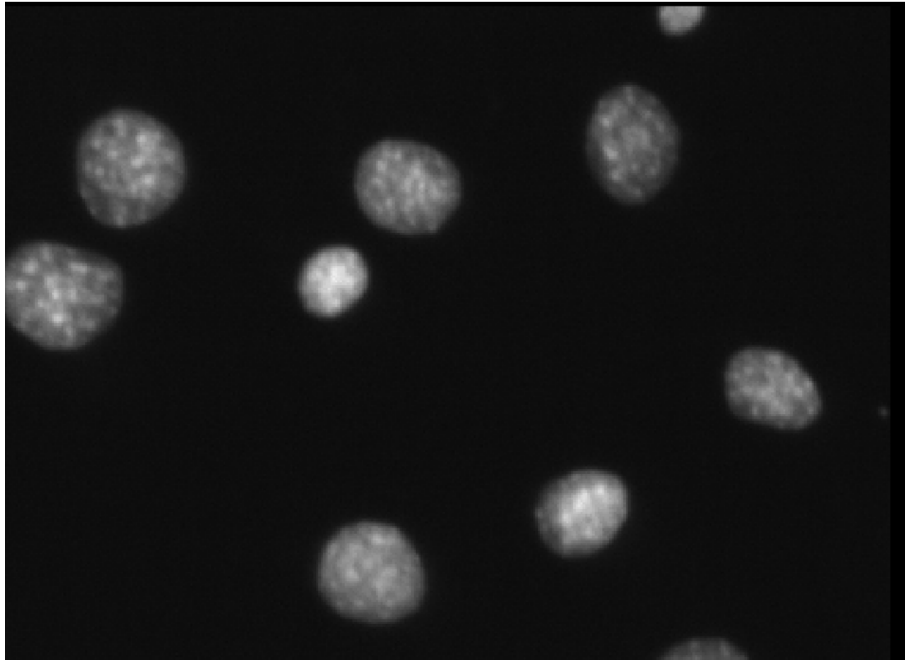
1. **Pre-processing** – noise removal (Unit 2), contrast enhancement.
2. **Thresholding** – separate foreground from background.
3. **Binary operations** – refine binary mask (morphological operations).
4. **Connected-component labelling** – identify individual objects.
5. **Post-processing / measurement** – filter by size, compute features.

1. Basic Concepts
- 2. Thresholding**
3. Binary Operations
4. Connected Components and Labelling
5. Watershed
6. Evaluation of Segmentation Quality
7. Readings and References

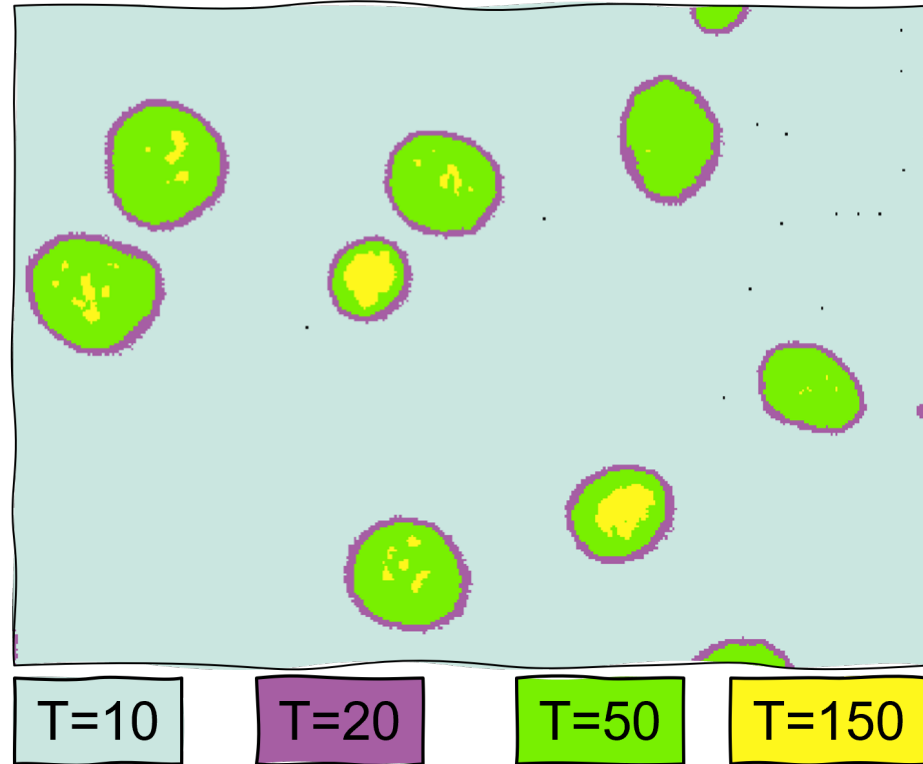
Definition. Given a threshold $T \in [I_{\min}, I_{\max}]$, the binary image is:

$$B[x, y] = \begin{cases} 1 & \text{if } I[x, y] \geq T \\ 0 & \text{otherwise} \end{cases}$$

- Simplest segmentation method.
- Works well when foreground and background have clearly separated intensity distributions.
- Fails with uneven illumination.



Intensity image



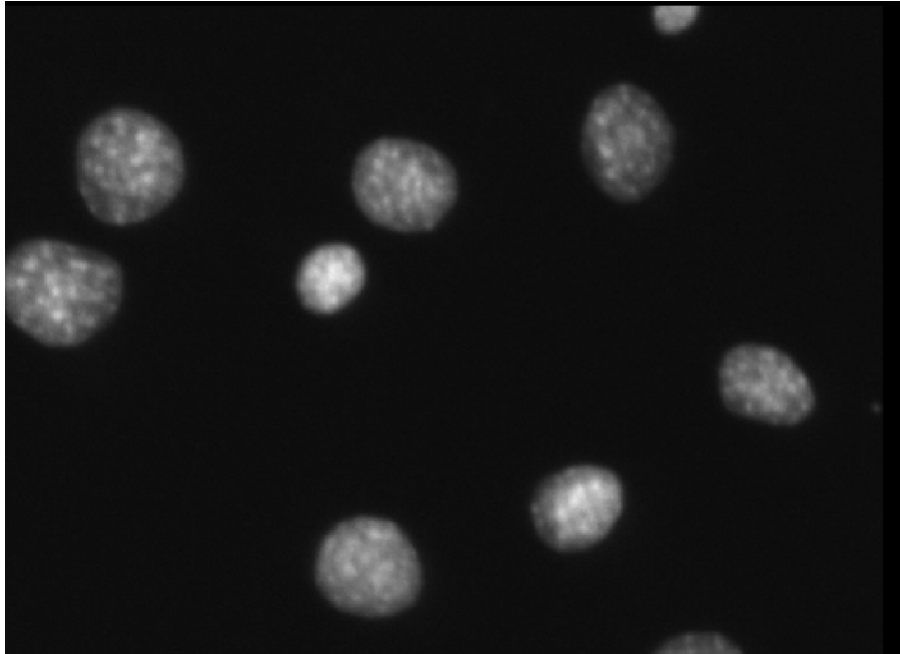
Nucleus image from [1].

Key idea: choose T^* that *minimises the intra-class variance* (equivalently, maximises the inter-class variance).

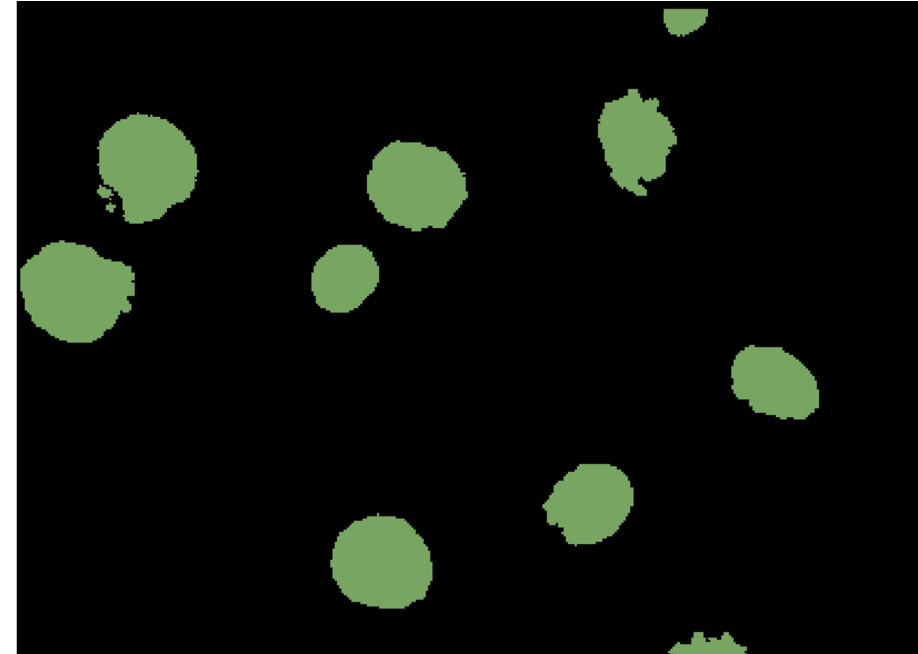
Let $\omega_0(T)$, $\omega_1(T)$ be the class probabilities and $\mu_0(T)$, $\mu_1(T)$ the class means. Then:

$$\sigma_B^2(T) = \omega_0(T) \omega_1(T) [\mu_0(T) - \mu_1(T)]^2$$
$$T^* = \arg \max_T \sigma_B^2(T)$$

- Computed from the normalised histogram $p[k]$ (Unit 1).
- Optimal for bi-modal intensity distributions.
- `skimage.filters.threshold_otsu`

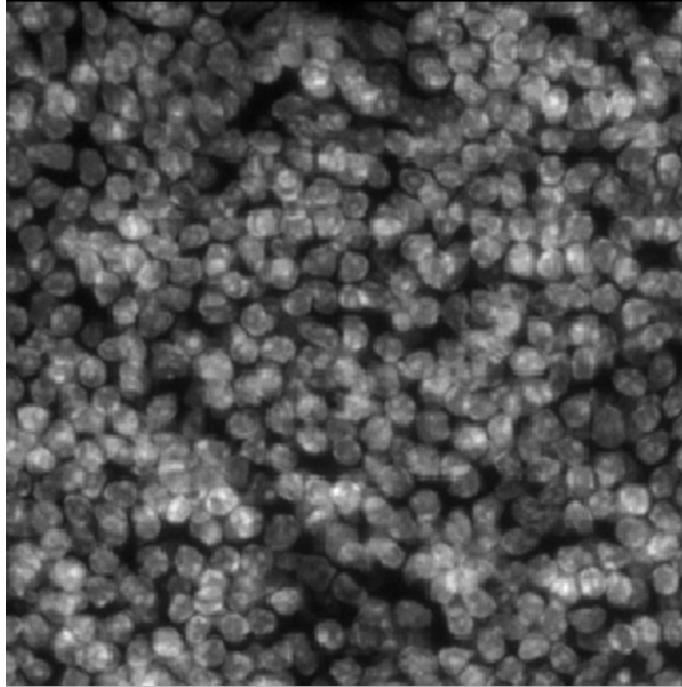


Intensity image

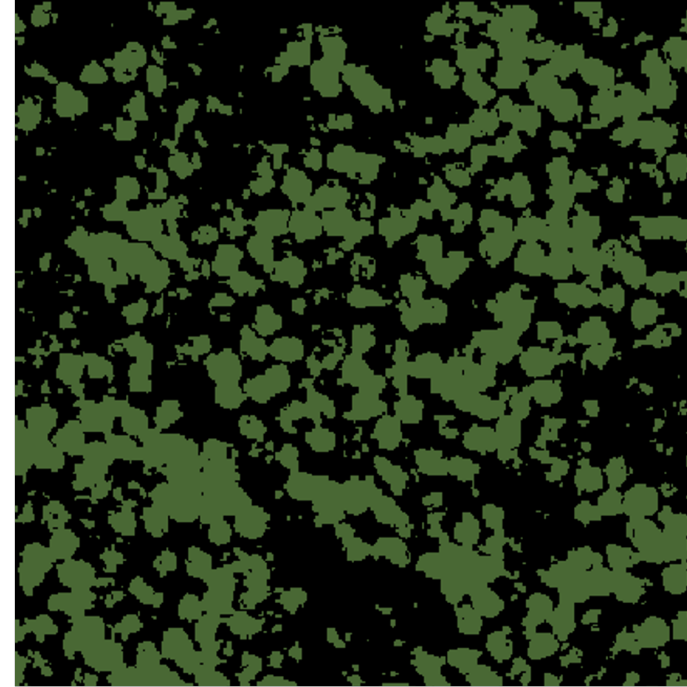


$T_{\text{OTSU}}=62$

Nucleus image from [1].



Intensity image



$T_{\text{OTSU}}=86$

Nucleus image Daniela Weidner.

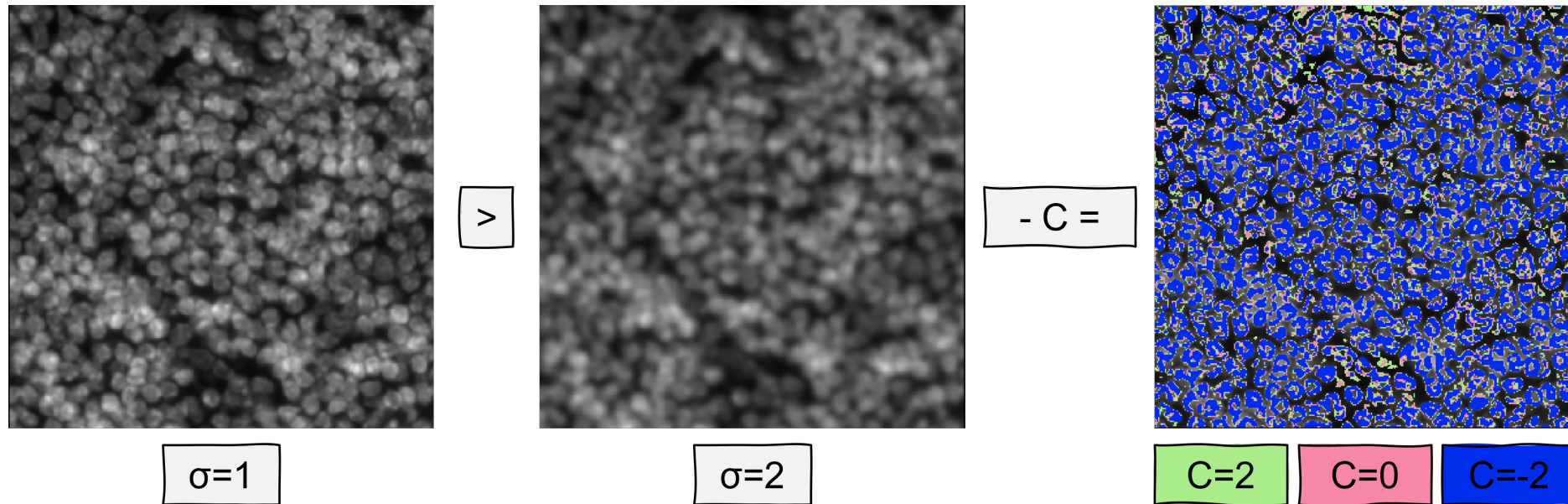
- **Problem:** uneven illumination makes a single T insufficient.
- **Solution:** compute a spatially varying threshold $T[x, y]$.

$$T[x, y] = \mu_{\mathcal{N}(x,y)} - c$$

where $\mu_{\mathcal{N}(x,y)}$ is the local mean over a neighbourhood $\mathcal{N}$ and c is a constant offset.

- Variants: Gaussian-weighted local mean, median-based, Sauvola.
- `skimage.filters.threshold_local`
- **Trade-off:** window size controls scale sensitivity vs. noise robustness.

Local (Adaptive) Thresholding



Nucleus image Daniela Weidner.

1. Basic Concepts
2. Thresholding
- 3. Binary Operations**
4. Connected Components and Labelling
5. Watershed
6. Evaluation of Segmentation Quality
7. Readings and References

Morphological operations transform a binary image B using a *structuring element* S (a small binary shape, e.g. a disk of radius r).

The two fundamental operations are:

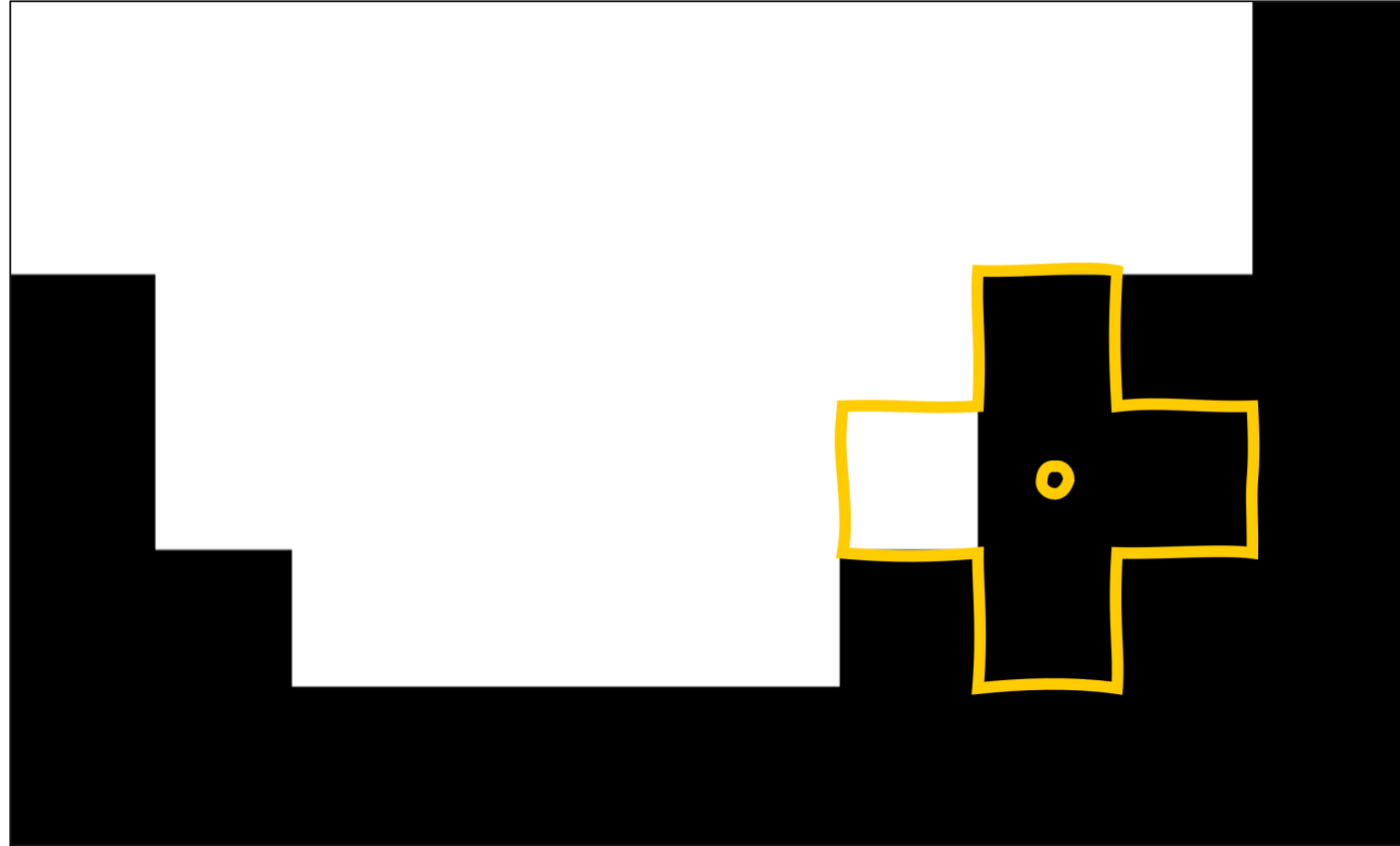
- **Erosion** $B \ominus S$: a pixel is 1 iff S fits entirely inside B .
- **Dilation** $B \oplus S$: a pixel is 1 iff S overlaps B at least once.

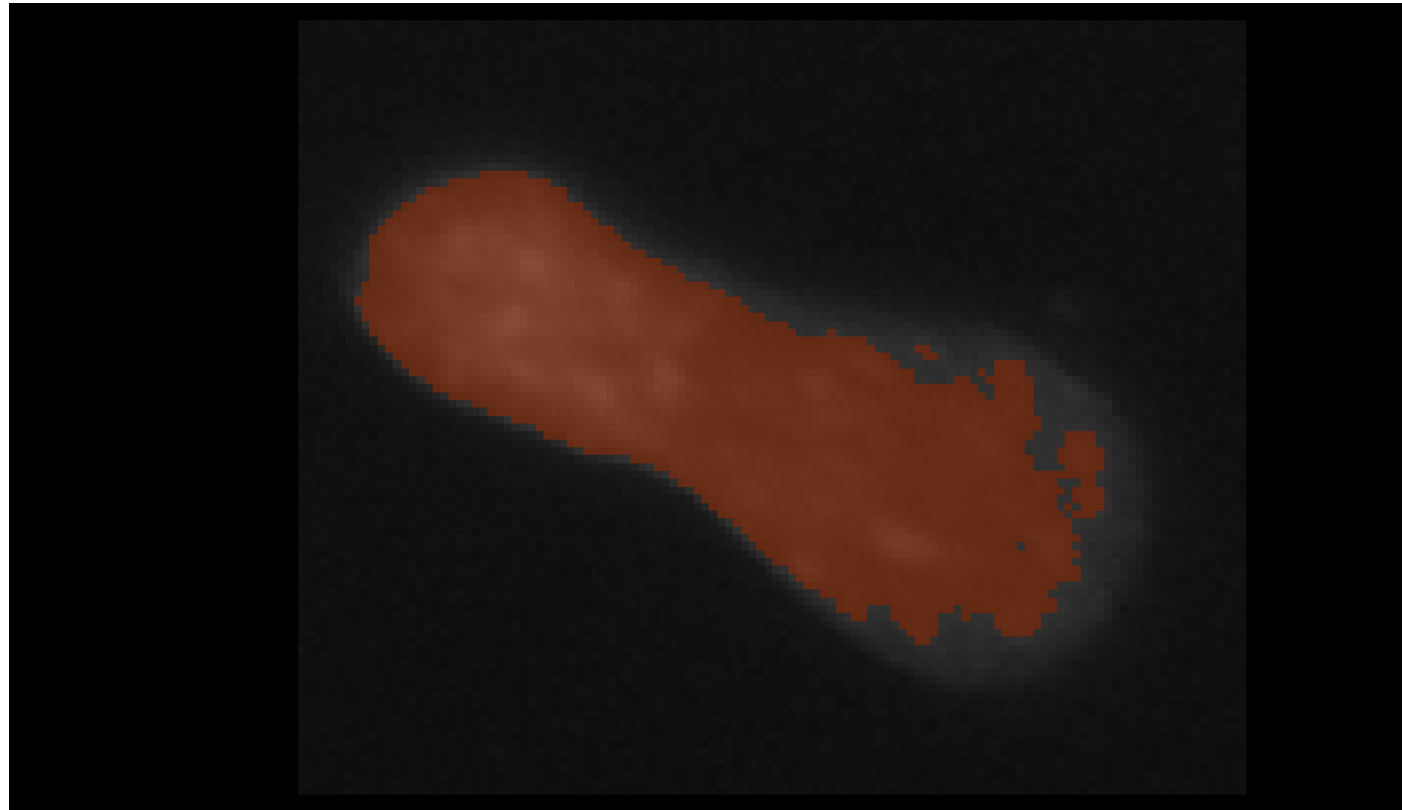
Formal definition (erosion):

$$(B \ominus S)[x, y] = \min_{(s_x, s_y) \in S} B[x + s_x, y + s_y]$$

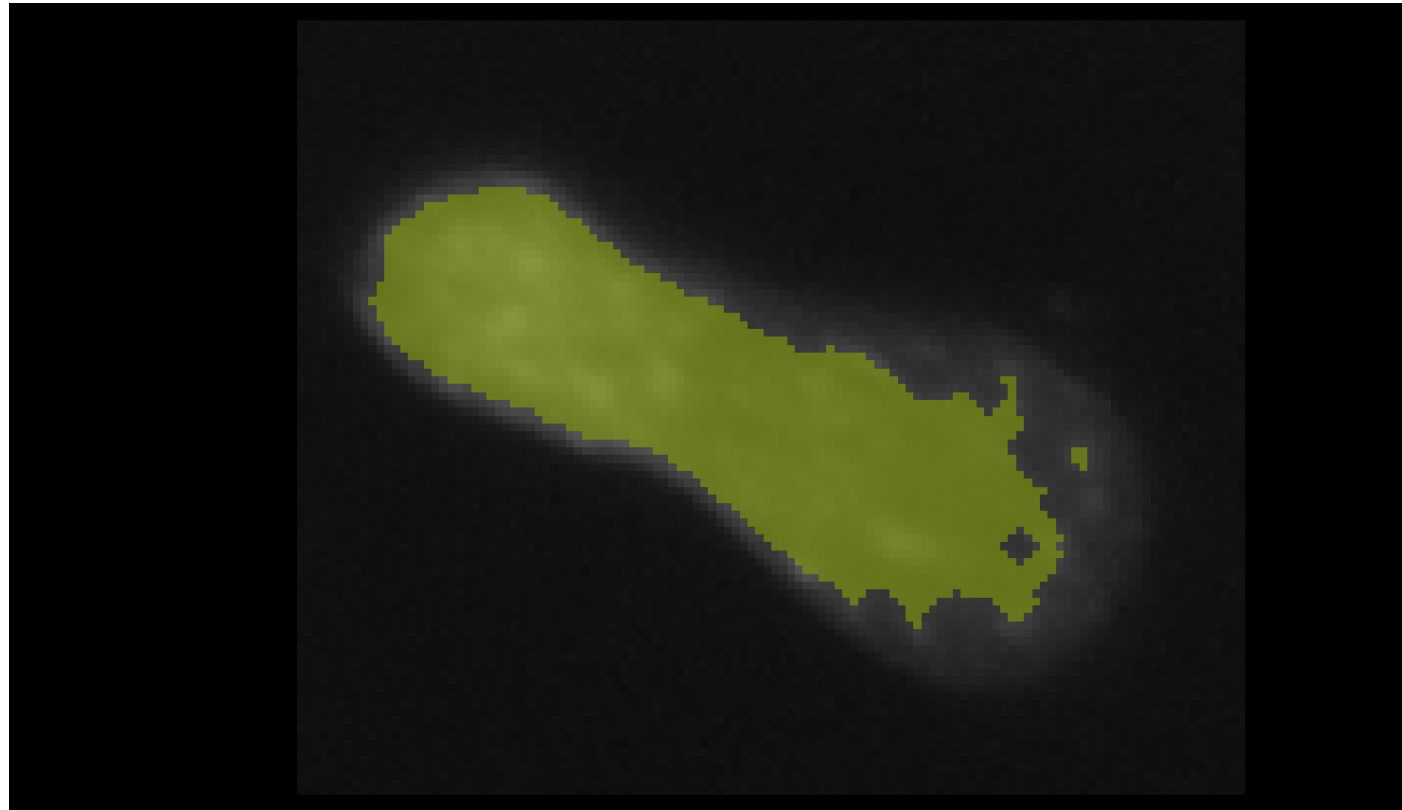
Formal definition (dilation):

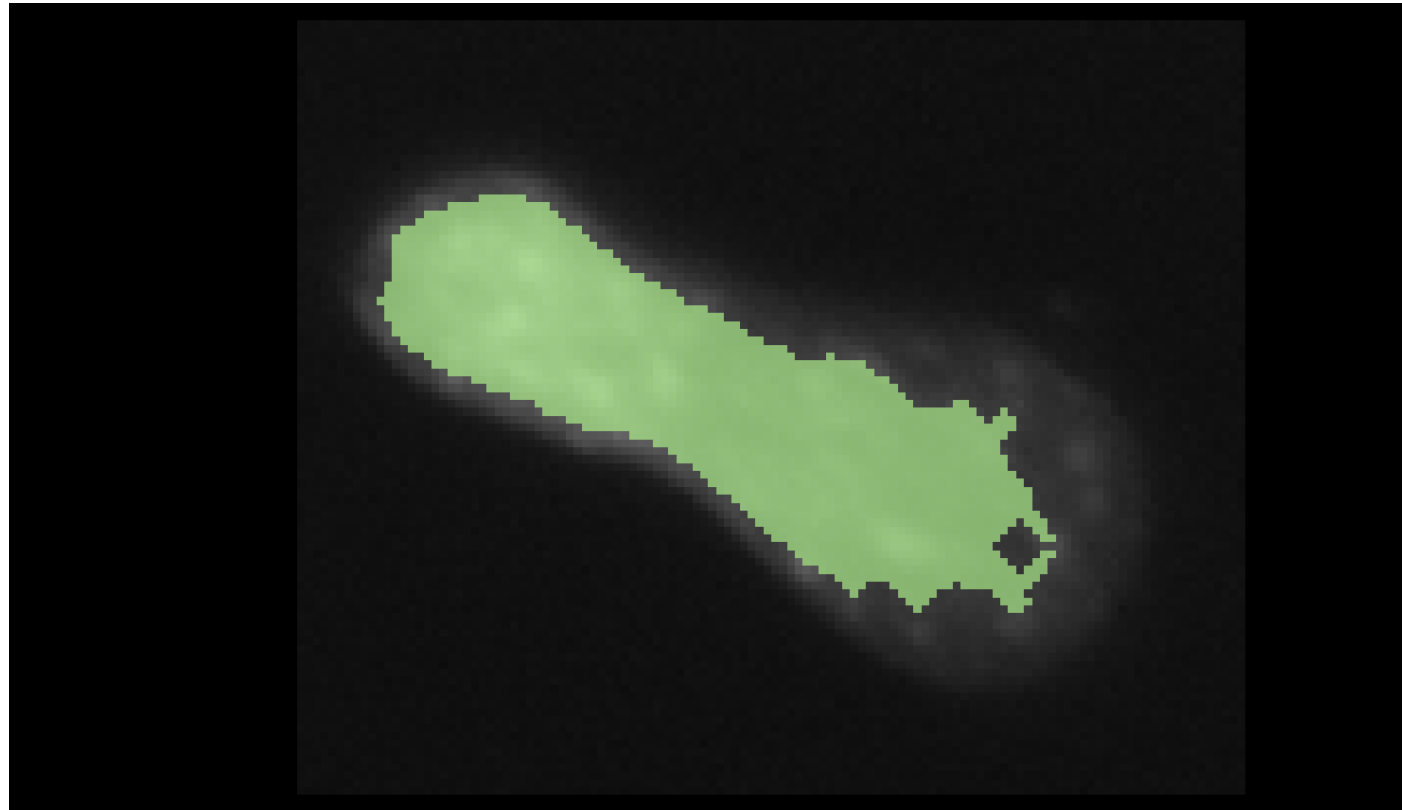
$$(B \oplus S)[x, y] = \max_{(s_x, s_y) \in S} B[x - s_x, y - s_y]$$

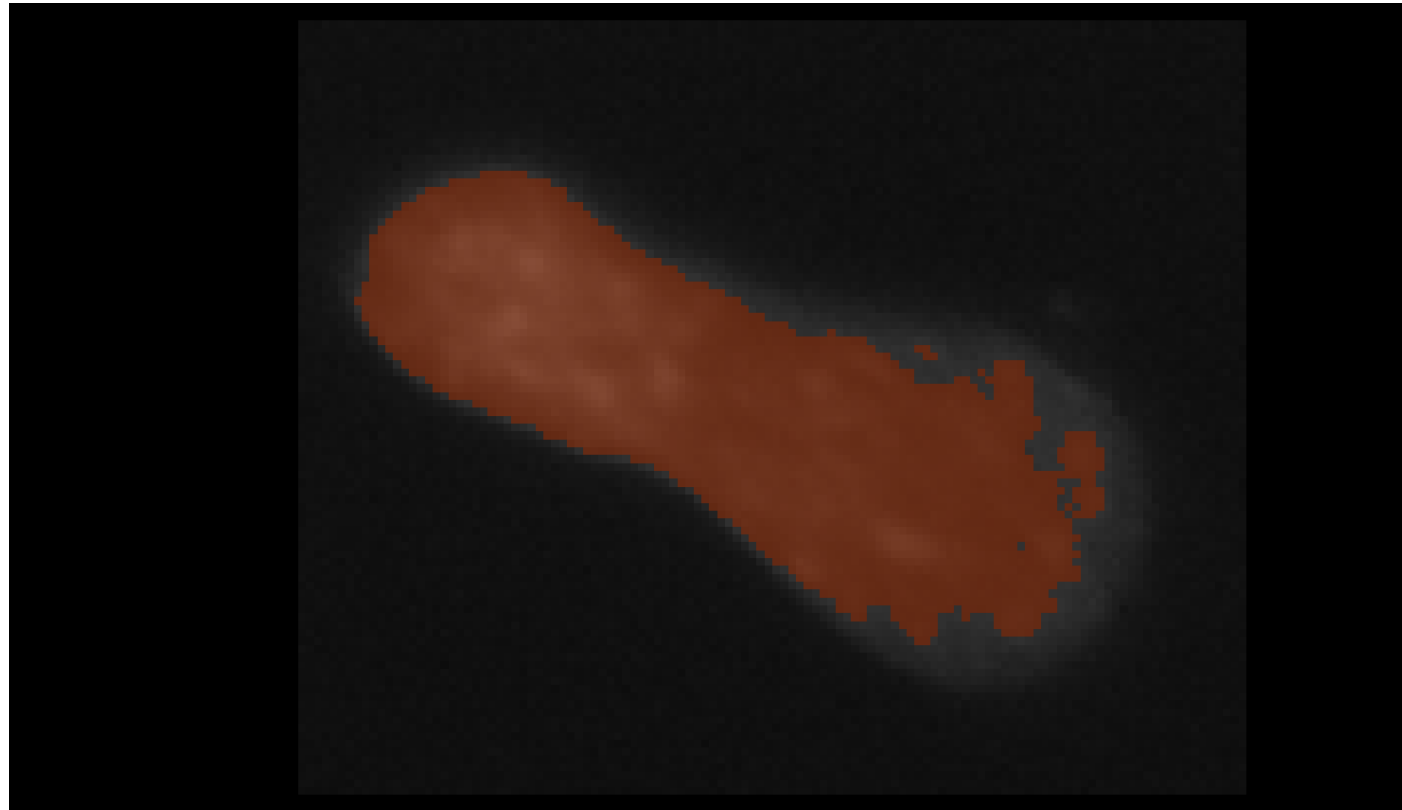


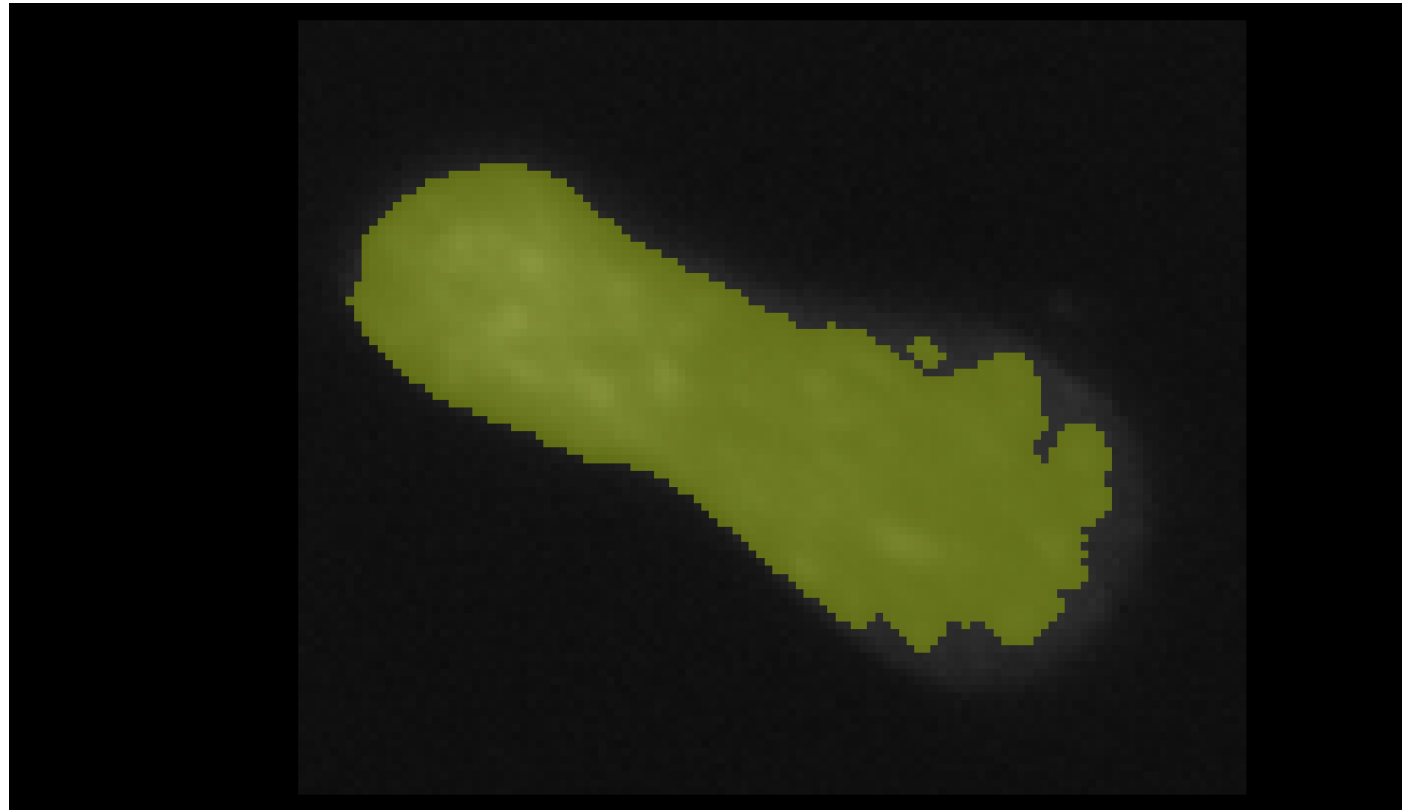


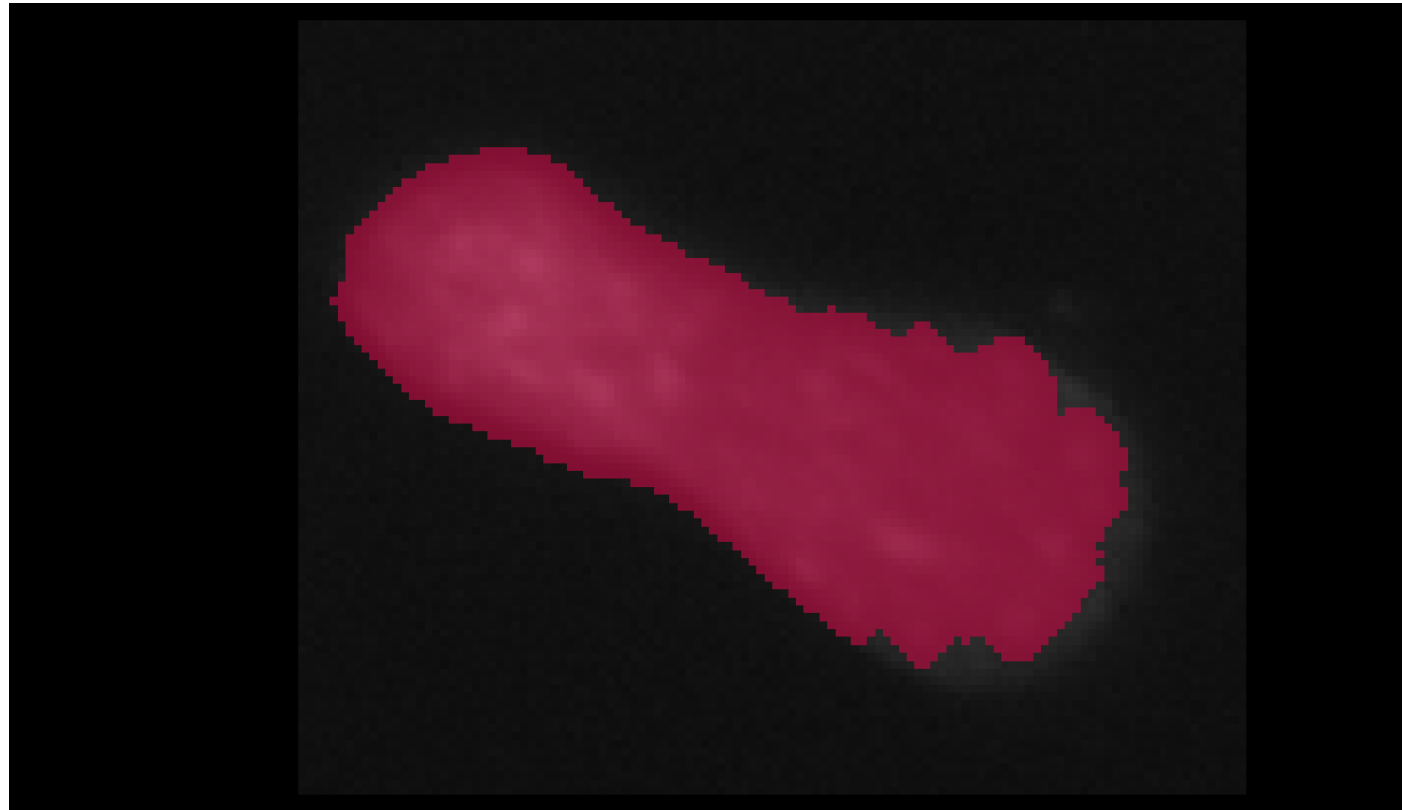


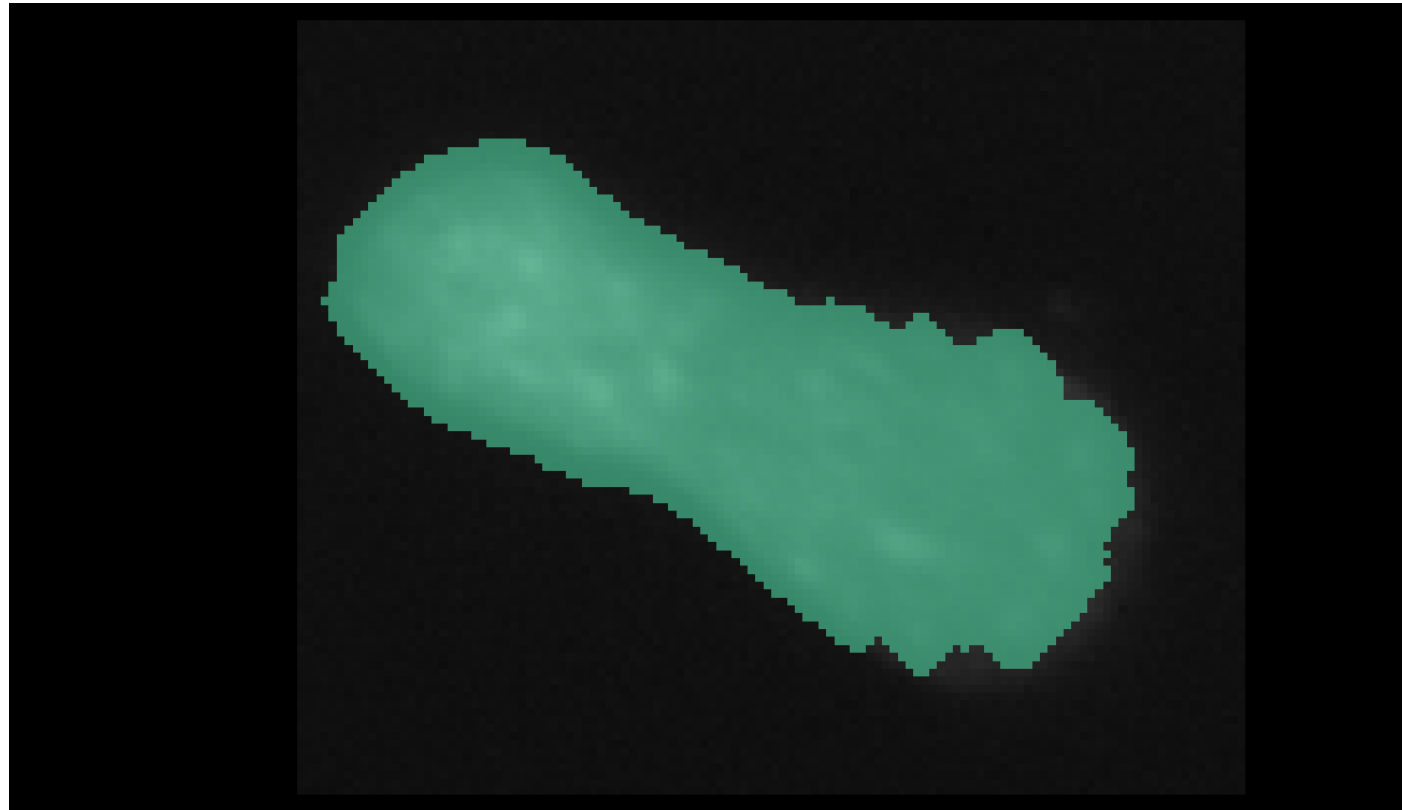












Erosion $B \ominus S$

- Shrinks foreground objects.
- Removes small isolated pixels (noise).
- Can disconnect touching objects.
- Enlarges holes.

Dilation $B \oplus S$

- Expands foreground objects.
- Closes small holes.
- Can merge nearby objects.
- Fills boundary gaps.

Opening – erosion followed by dilation:

$$B \circ S = (B \ominus S) \oplus S$$

- Removes small bright objects (noise) without strongly affecting larger objects.
- Smooths object boundaries.

Closing – dilation followed by erosion:

$$B \bullet S = (B \oplus S) \ominus S$$

- Fills small holes and gaps within objects.
- Connects nearby objects (if gap < structuring element).

Morphological gradient – highlights edges:

$$G = (B \oplus S) - (B \ominus S)$$

White top-hat transform – extracts small bright structures:

$$W = B - (B \circ S)$$

Black top-hat transform – extracts small dark structures:

$$K = (B \bullet S) - B$$

- Top-hat transforms are useful for correcting uneven background.

- Shape: disk (isotropic), square, cross, line (directional).
- Size r : determines the scale of features affected.
 - Too small: noise not removed.
 - Too large: true structures eroded away.
- **Rule of thumb:** r should be smaller than the smallest object of interest but larger than the noise features to be removed.
- In skimage: `skimage.morphology.disk(r)`, `square(r)`, ...

1. Basic Concepts
2. Thresholding
3. Binary Operations
- 4. Connected Components and Labelling**
5. Watershed
6. Evaluation of Segmentation Quality
7. Readings and References

Definition. Two foreground pixels p and q are *connected* if there is a path of foreground pixels linking them, where adjacency is defined by the chosen connectivity.

- **4-connectivity** (2D): a pixel is adjacent to its 4 orthogonal neighbours (up, down, left, right).
- **8-connectivity** (2D): a pixel is adjacent to all 8 neighbours (including diagonals).
- In 3D: **6-connectivity**, **18-connectivity**, **26-connectivity**.

Goal: assign a unique integer label to each connected component of the binary image.

Two-pass algorithm (classical):

1. *First pass* – scan raster order; assign provisional labels and record *equivalences* when two labels meet.
2. Merge equivalent labels.
3. *Second pass* – replace provisional labels with final labels.

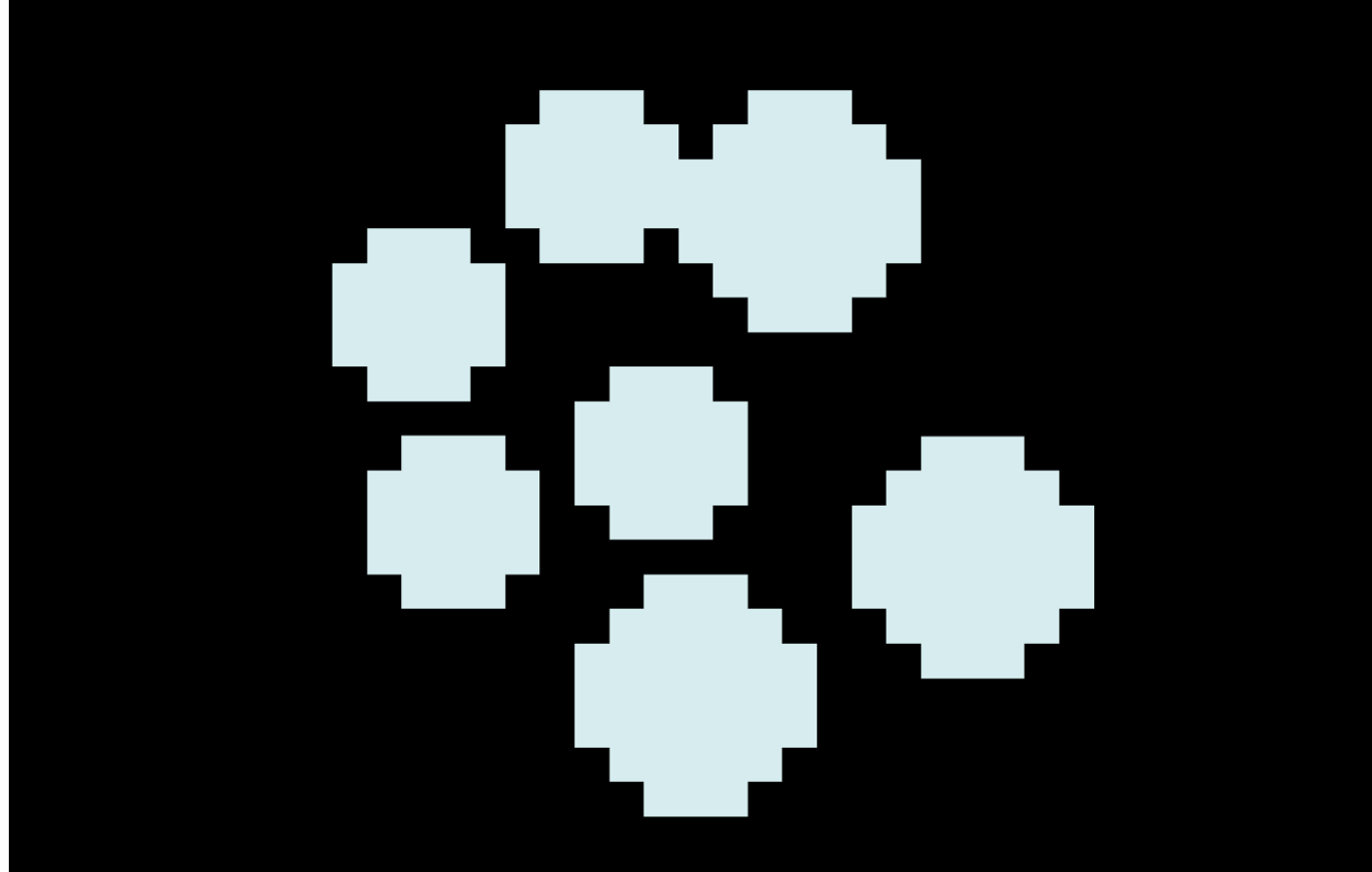
Output: label image $L[x, y] \in \{0, 1, 2, \dots, N_{\text{obj}}\}$, where 0 denotes background and $1, \dots, N_{\text{obj}}$ denote individual objects.

`skimage.measure.label`, `scipy.ndimage.label`

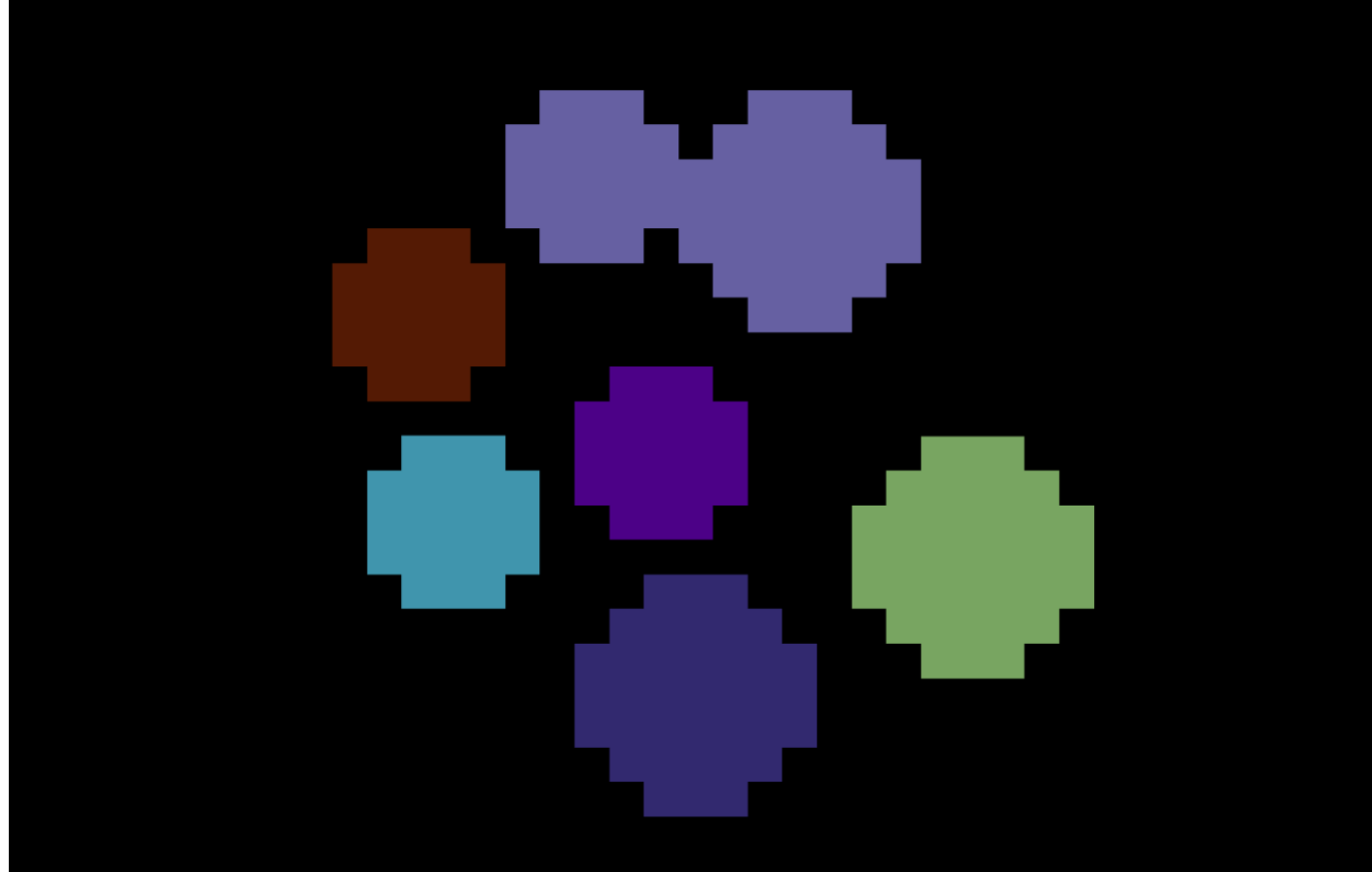
1. Basic Concepts
2. Thresholding
3. Binary Operations
4. Connected Components and Labelling
- 5. Watershed**
6. Evaluation of Segmentation Quality
7. Readings and References

- Connected-component labelling assigns one label per connected component.
- **Problem:** touching or overlapping objects form a single connected component $\Rightarrow$ they receive the same label.
- Example: densely packed nuclei that share a membrane contact appear as one large blob.
- **Solution:** the *watershed transform* uses the image topology to find natural boundaries between touching objects.

Motivation: The Touching-Object Problem



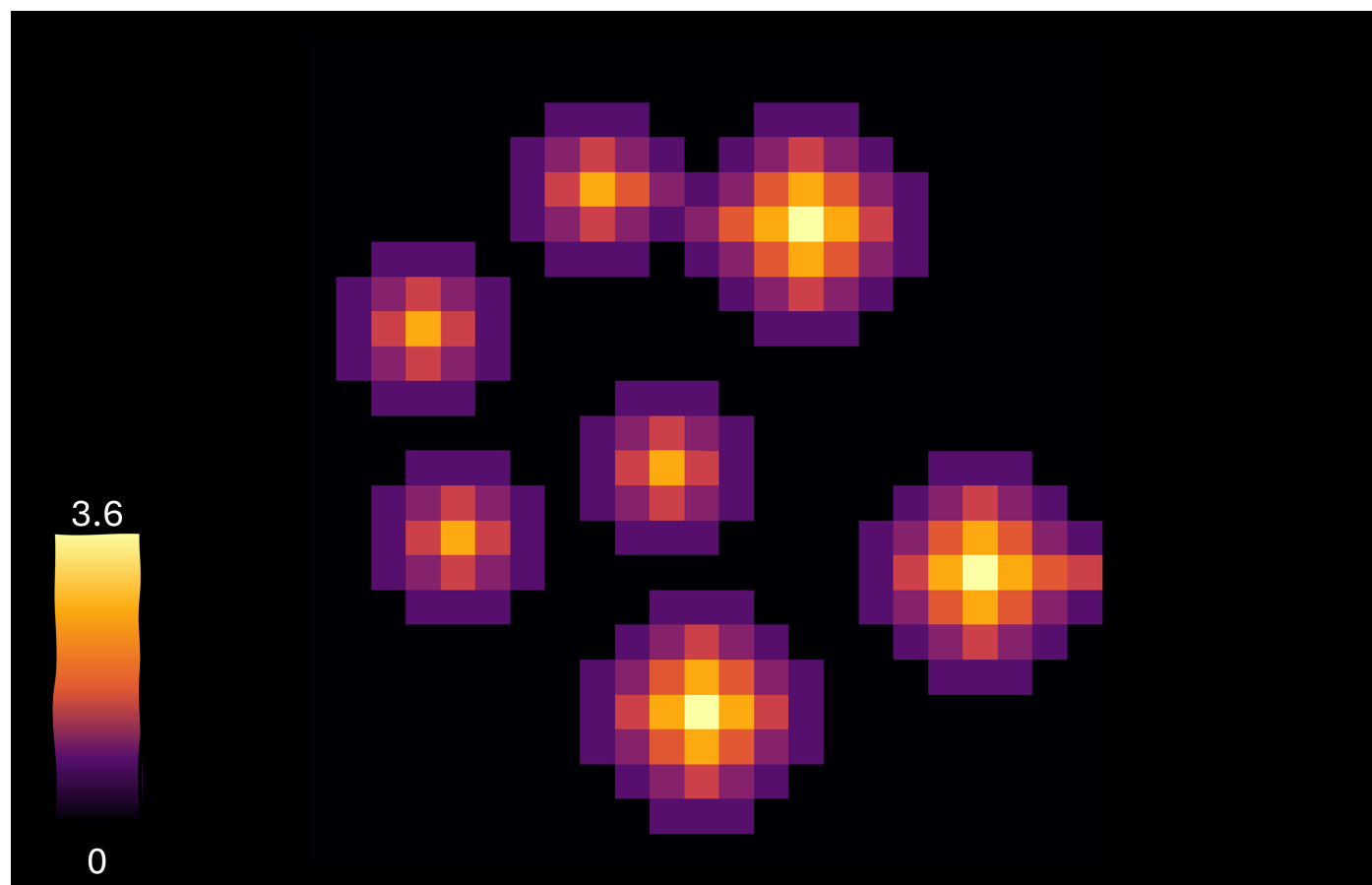
Motivation: The Touching-Object Problem



Definition. For a binary image B , the *distance transform* D assigns to each foreground pixel its distance to the nearest background pixel:

$$D[x, y] = \min_{(x', y') : B[x', y'] = 0} \sqrt{(x - x')^2 + (y - y')^2}$$

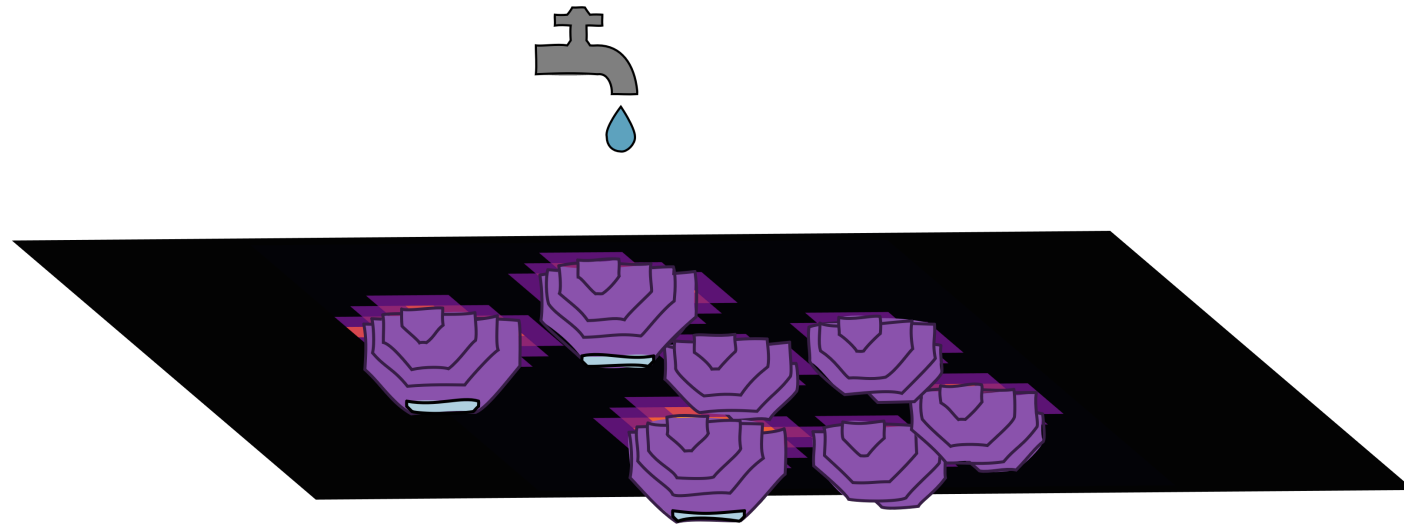
- Local maxima of D correspond to the *medial axis* (skeleton) of the objects.
- The value at each local maximum approximates the **radius** of the inscribed circle.
- `scipy.ndimage.distance_transform_edt`



- Interpret the image $f[x, y]$ (or a derived map) as a **topographic surface**: high intensity = high altitude.
- Imagine rain falling uniformly on the surface.
- Water flows downhill and accumulates in *catchment basins* (valleys).
- The *watershed lines* are the ridges separating adjacent basins.
- In practice the surface is **inverted** so that object centres (bright regions) become *valleys* that fill up first.

$$f_{\text{ws}}[x, y] = -f[x, y] \quad (\text{inverted surface})$$

The Topographic Metaphor

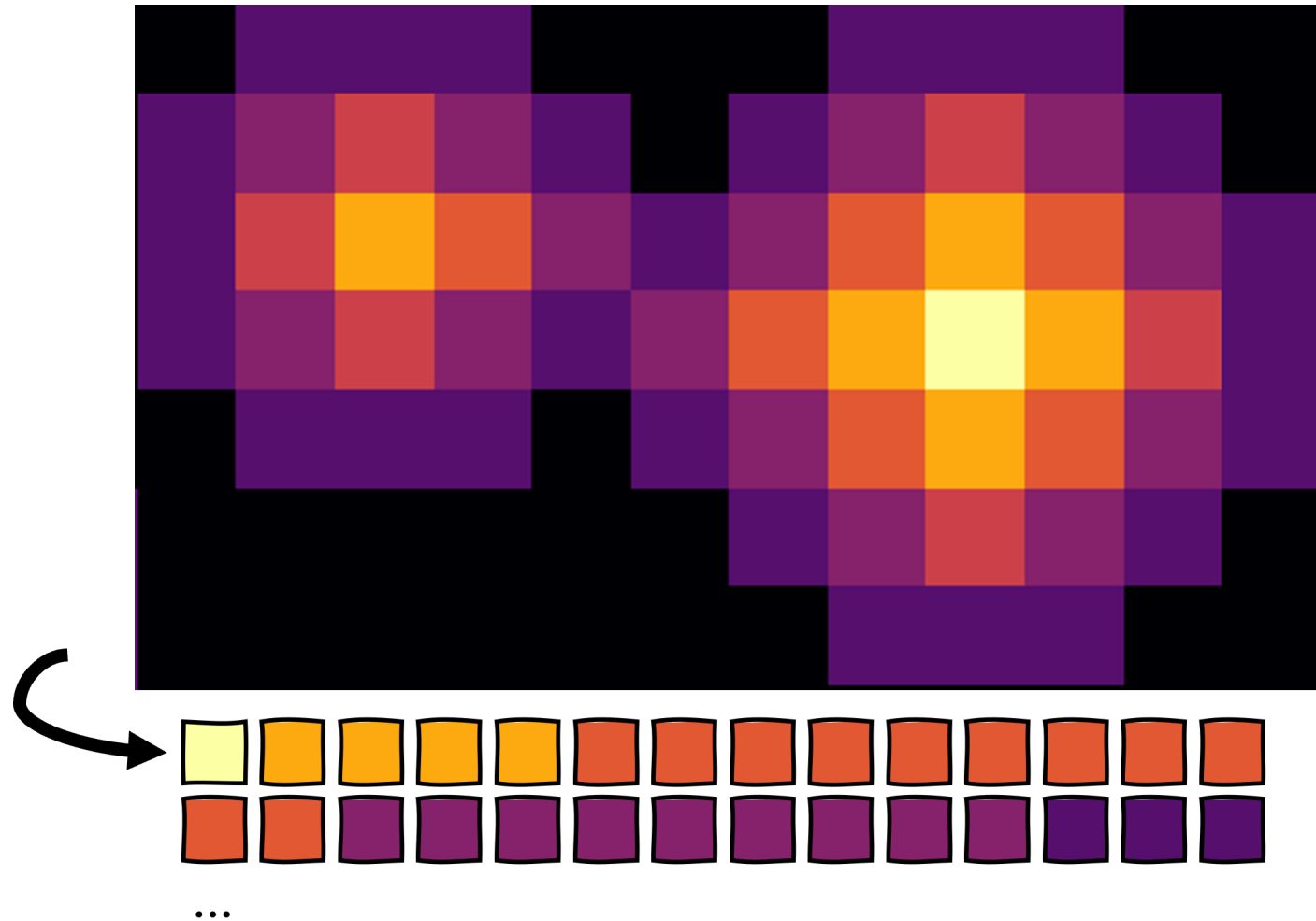


Input: a grayscale surface f (typically an inverted distance map or gradient image).

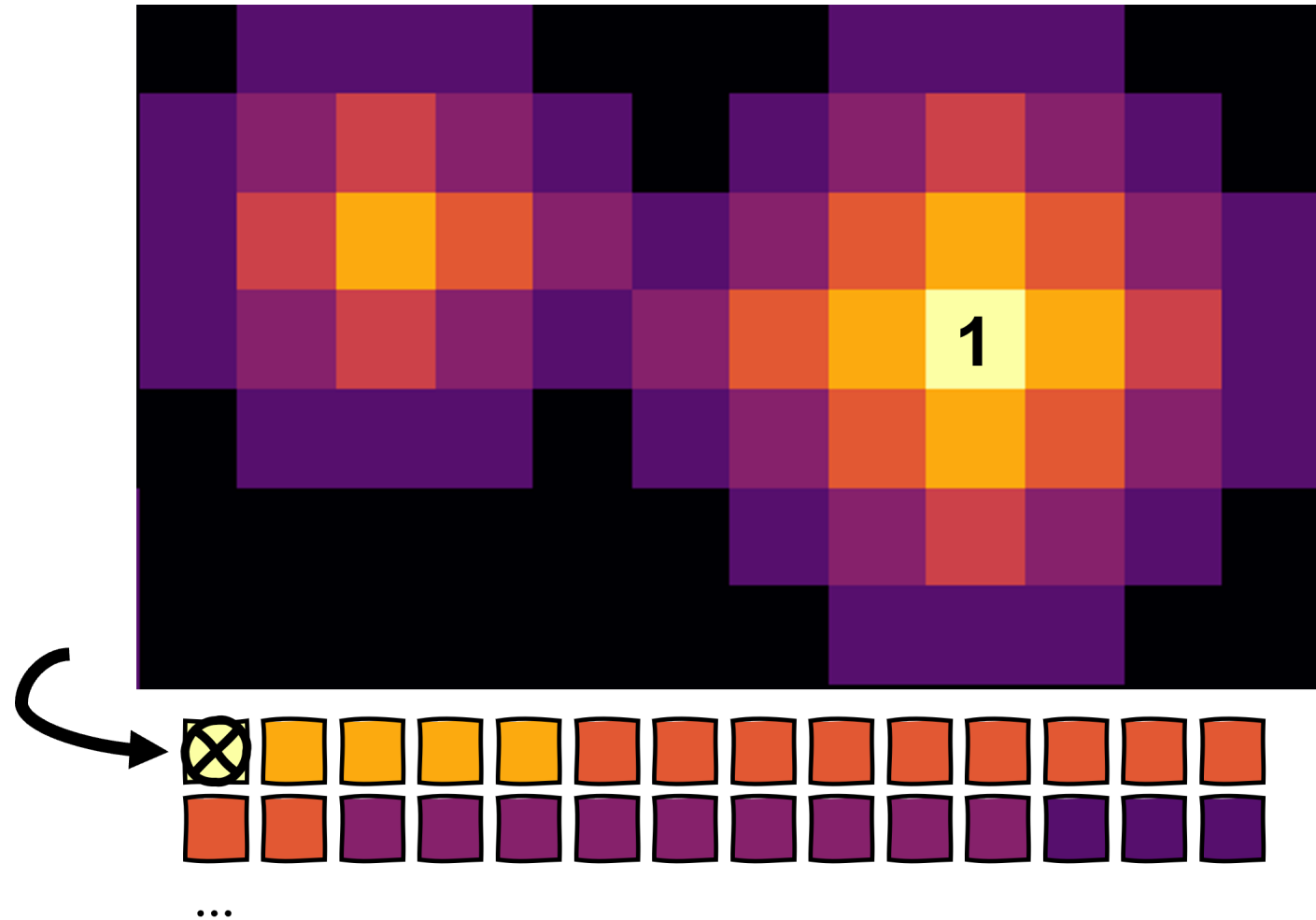
Iterative flooding procedure:

1. Sort all pixels by ascending intensity (priority queue / min-heap).
2. Process pixels from lowest to highest value:
 - If all labelled neighbours belong to the *same* basin k : assign pixel to basin k .
 - If labelled neighbours belong to *different* basins: mark pixel as **watershed line** ($L = 0$).
 - If no labelled neighbours exist: start a *new* basin.
3. Continue until all pixels are assigned.

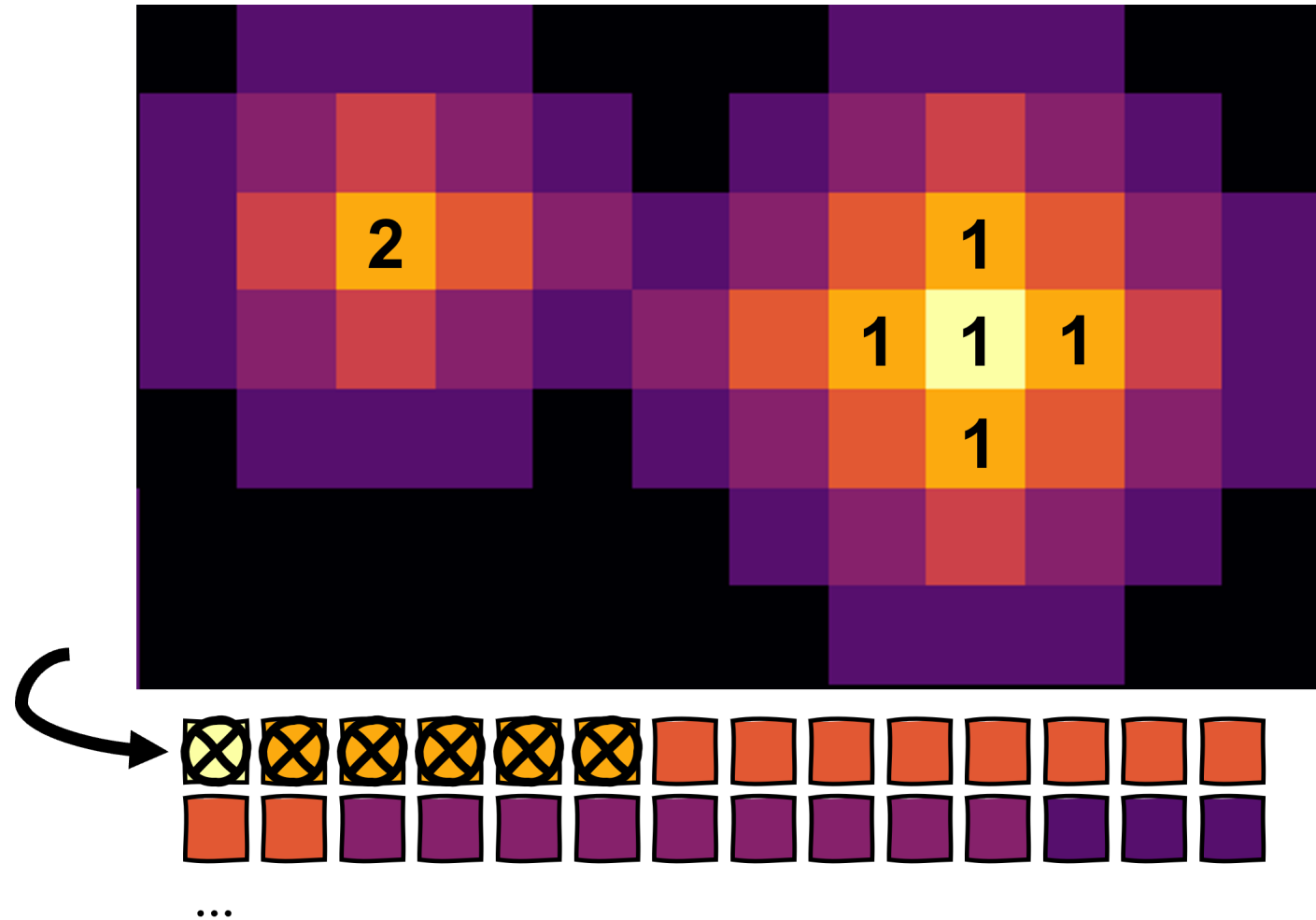
Watershed Algorithm – Flood-Fill Formulation

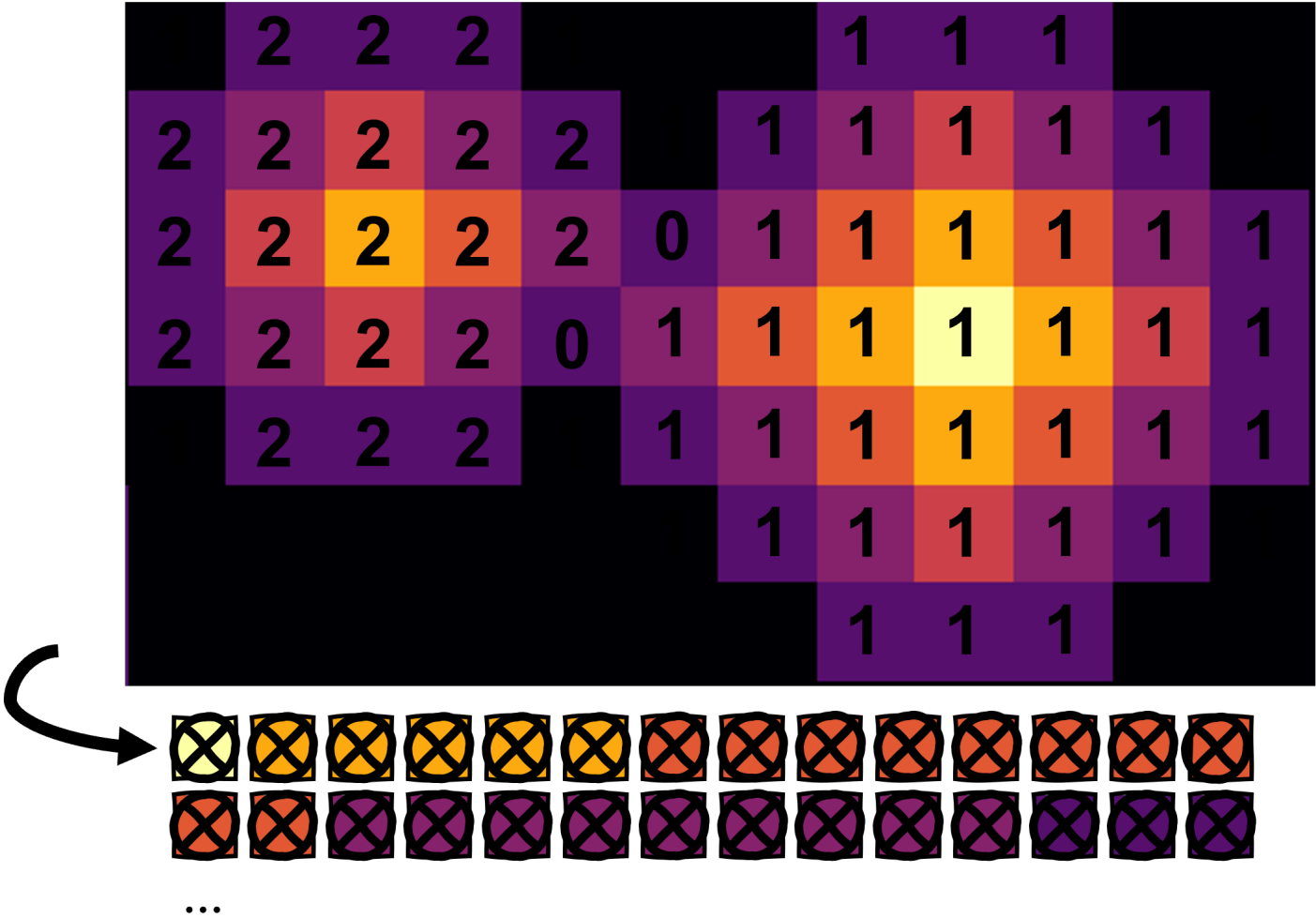


Watershed Algorithm – Flood-Fill Formulation



Watershed Algorithm – Flood-Fill Formulation



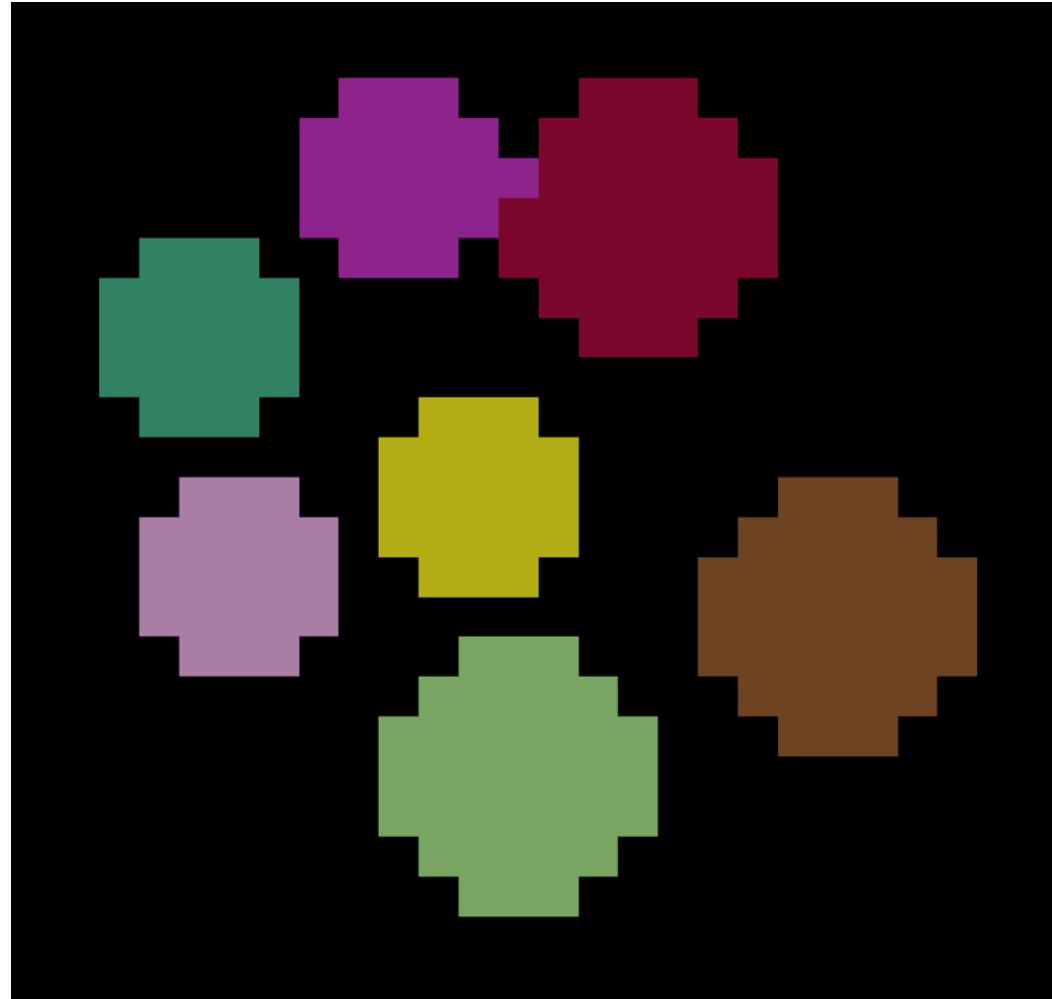


- When no prior knowledge about object locations is available, **local maxima** of the surface serve as automatic seeds.
- A pixel p is a local maximum if

$$f[p] \geq f[q] \quad \forall q \in \mathcal{N}(p)$$

- **Standard workflow (distance-map watershed):**
 1. Threshold $\rightarrow$ binary mask B .
 2. Compute Euclidean distance transform D of B .
 3. Detect local maxima of D (peaks = object centres).
 4. Run watershed on $-D$, each local maximum seeds one basin.
- **Problem:** noise creates *spurious* local maxima $\Rightarrow$ over-segmentation.
- **Remedy:** smooth D before peak detection, or provide markers.
`skimage.morphology.local_maxima`, `skimage.feature.peak_local_max`

Watershed with Local Maxima as Seeds



- **From LoG / local maxima:** one marker per detected blob centre.
- **From erosion:** heavily erode the binary mask; remaining pixels are guaranteed to be interior points of each object.
- **From manual annotation:** user places one marker per object (semi-supervised).
- **From foreground / background markers:** two-class seeded watershed (foreground seed + background seed) for simple binary segmentation.

Common pitfalls:

- Missing marker $\Rightarrow$ object merged with neighbour (under-segmentation).
- Extra marker $\Rightarrow$ one object split into two (over-segmentation).
- Markers must lie *strictly inside* the objects (not on boundaries).

Recommended pipeline for dense, touching nuclei:

1. **Pre-processing:** Gaussian smoothing ($\sigma \approx 1-2$ px).
2. **Thresholding:** Otsu $\rightarrow$ binary mask B .
3. **Binary refinement:** opening to remove debris, fill holes.
4. **Distance transform:** $D = \text{EDT}(B)$.
5. **Seed detection:** LoG on I or local maxima of D .
6. **Seeded watershed:** $\text{watershed}(-D, M, \text{mask}=B)$, where M is the marker image.
7. **Post-processing:** `regionprops` $\rightarrow$ filter by area, eccentricity, border exclusion.

Note: the `mask` parameter restricts flooding to foreground pixels, preventing background regions from being labelled.

1. Basic Concepts
2. Thresholding
3. Binary Operations
4. Connected Components and Labelling
5. Watershed
- 6. Evaluation of Segmentation Quality**
7. Readings and References

Given a binary prediction $\hat{B}$ and ground truth B^* :

$$\text{TP} = \sum_{x,y} \hat{B}[x,y] \cdot B^*[x,y], \quad \text{FP} = \sum_{x,y} \hat{B}[x,y] \cdot (1 - B^*[x,y])$$

$$\text{FN} = \sum_{x,y} (1 - \hat{B}[x,y]) \cdot B^*[x,y]$$

Metric	Formula
Precision	$\text{TP} / (\text{TP} + \text{FP})$
Recall (Sensitivity)	$\text{TP} / (\text{TP} + \text{FN})$
Dice coefficient (F1)	$2 \text{TP} / (2 \text{TP} + \text{FP} + \text{FN})$
Jaccard index (IoU)	$\text{TP} / (\text{TP} + \text{FP} + \text{FN})$

Note: $\text{Dice} = \frac{2 \text{IoU}}{1 + \text{IoU}}$.

- Pixel-level metrics do not distinguish individual objects – an object can be partly correct and still yield a high IoU.
- **Average Precision (AP)** at IoU threshold τ :
 1. For each predicted object, compute IoU with the best-matching ground-truth object.
 2. Predicted object is TP if $\text{IoU} \geq \tau$, FP otherwise.
 3. $\text{AP}(\tau) = \text{TP} / (\text{TP} + \text{FP} + \text{FN})$
- **Mean Average Precision (mAP)**: average AP over multiple τ , e.g. $\tau \in \{0.5, 0.55, \dots, 0.95\}$.
- Standard benchmark metric for cell segmentation challenges (CellSeg, Neurips 2022).

1. Basic Concepts
2. Thresholding
3. Binary Operations
4. Connected Components and Labelling
5. Watershed
6. Evaluation of Segmentation Quality
- 7. Readings and References**

- Haase et al., "Bio-image Analysis Notebooks" (haesleinhuepf.github.io).
- scikit-image documentation ([skimage.measure](#), [skimage.morphology](#)).

[1] Weimiao Yu et al. "Ccdb: 6843, mus musculus, neuroblastoma". In: *Cell Image Library* 10 (2022), W9CCDB6843.